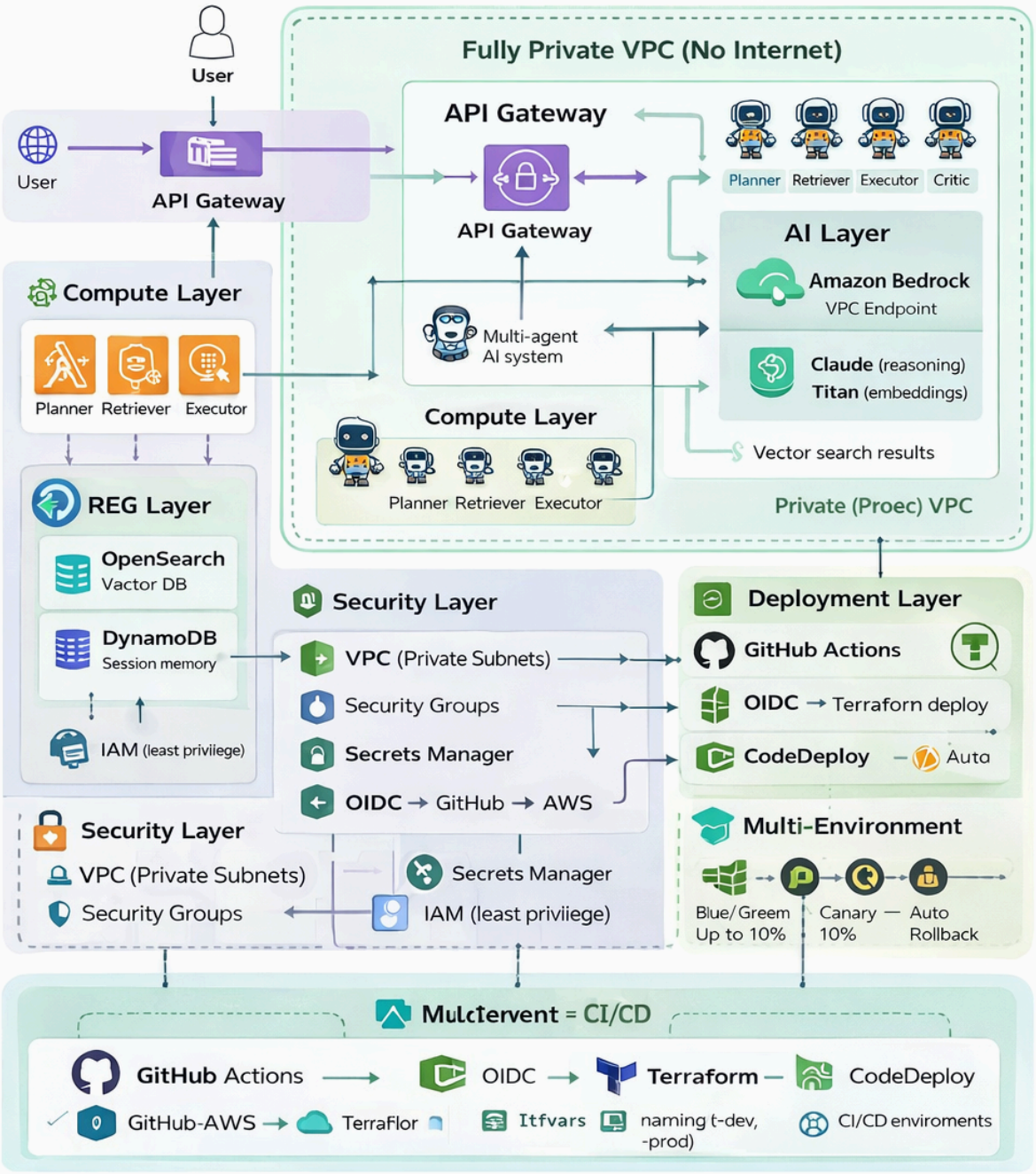
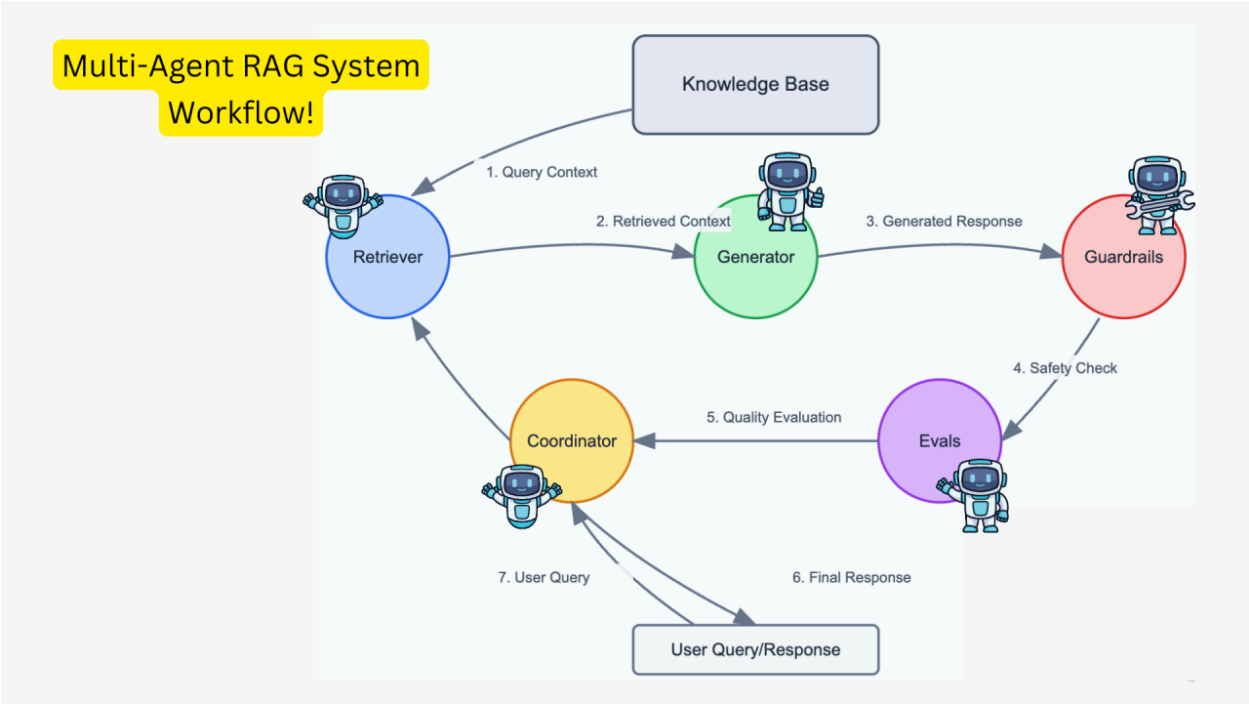
Final Enterprise Architecture

- Fully private VPC (no internet)
- Multi-agent AI system
- RAG with vector DB
- Bedrock LLM via VPC∠ endpoint
- CI/CD with GitHub OIDC
- Blue/Green deployment with auto rollback
- Multi-environment (dev/staging/prod)





Architecture Explanation

🌐 Entry Layer

- **API Gateway**
  - Public entry point
  - Routes `/agent` requests
  - Can be extended with JWT auth

⚙️ Compute Layer (Private)

- **Lambda (inside VPC)**
  - Multi-agent execution:
    - Planner
    - Retriever (RAG)
    - Executor
    - Critic
  - No internet access

🧠 AI Layer

- **Amazon Bedrock**
  - Claude (reasoning)
  - Titan (embeddings)
  - Access via **VPC Endpoint**

🔎 RAG Layer

- **OpenSearch (Vector DB)**
  - Private access only
  - Semantic search
  - Stores embeddings

🗄️ Data Layer

- **DynamoDB**
  - session memory
- **S3 (optional)**
  - document storage

🔐 Security Layer

- **VPC (Private Subnets)**
- **Security Groups**
- **Secrets Manager**
- **IAM (least privilege)**
- **OIDC (GitHub → AWS)**

🔁 Deployment Layer

- **CodeDeploy**
  - Blue/Green
  - Canary rollout (10%)
  - Auto rollback (CloudWatch alarm)

📊 Observability

- **CloudWatch**
  - logs + metrics
- **X-Ray**
  - tracing
- **Alarms**
  - trigger rollback

🔁 CI/CD Pipeline

- **GitHub Actions**
  - OIDC authentication
  - Terraform deploy
  - CodeDeploy trigger

🌍 Multi-Environment

- dev / staging / prod
- isolated via:
    - `tfvars`
    - naming (`-dev`, `-prod`)
    - CI/CD environments

🔄 End-to-End Flow

User → API Gateway

↓

Lambda (VPC)

↓

├ RAG → OpenSearch (vector search)

├ LLM → Bedrock (via VPC endpoint)

├ Memory → DynamoDB

└ Config → Secrets Manager

↓

Response

CI/CD:

GitHub → OIDC → Terraform → CodeDeploy → Lambda Alias

Deployment Flow:

Git Push

↓

GitHub Actions

↓

OIDC → AWS

↓

Terraform Apply

↓

New Lambda Version

↓

CodeDeploy (10% traffic)

↓

Monitor

↓

Auto rollback OR full rollout

🔐 What is OIDC in our Setup

Instead of storing secrets:

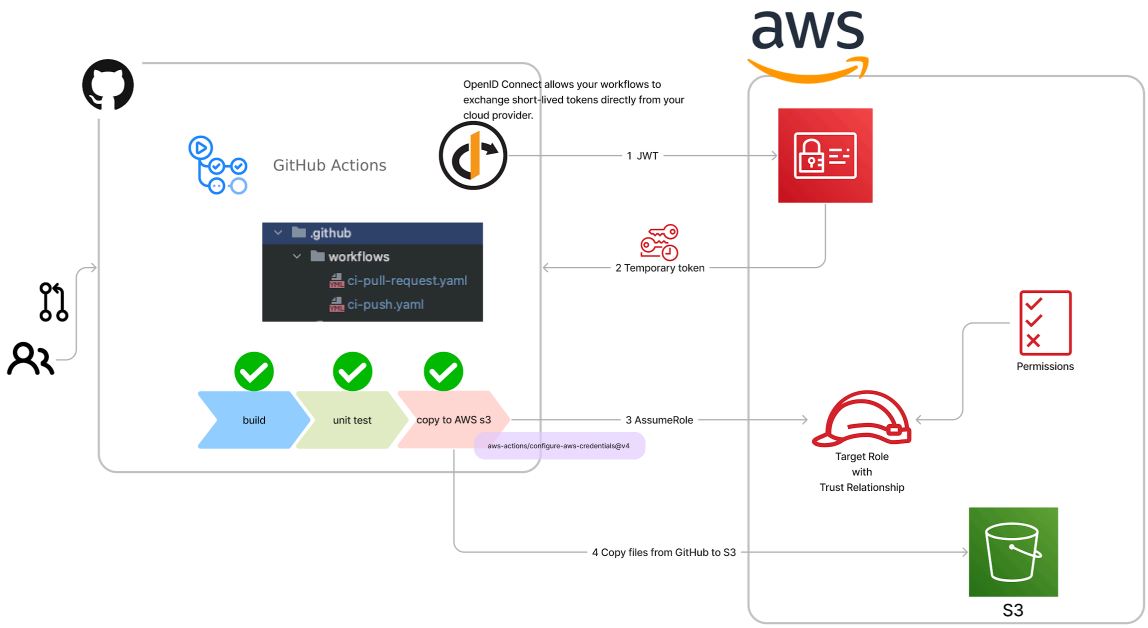
GitHub → AWS using ACCESS KEYS ❌

You do:

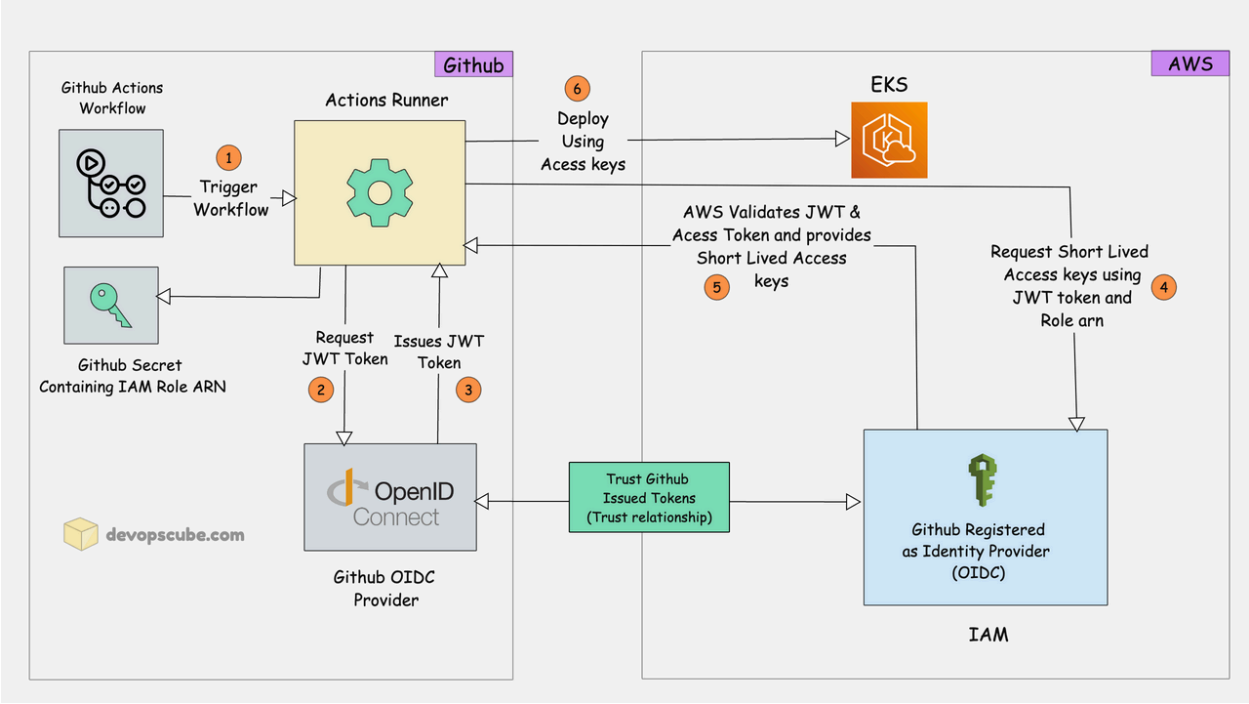
GitHub → OIDC token → Assume IAM Role → AWS ✅

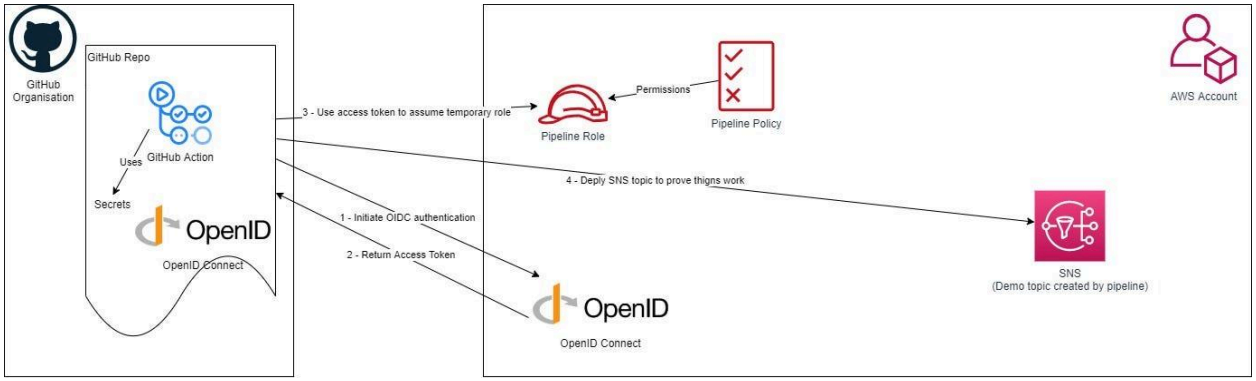
👉 No secrets stored
👉 Temporary credentials
👉 Much safer

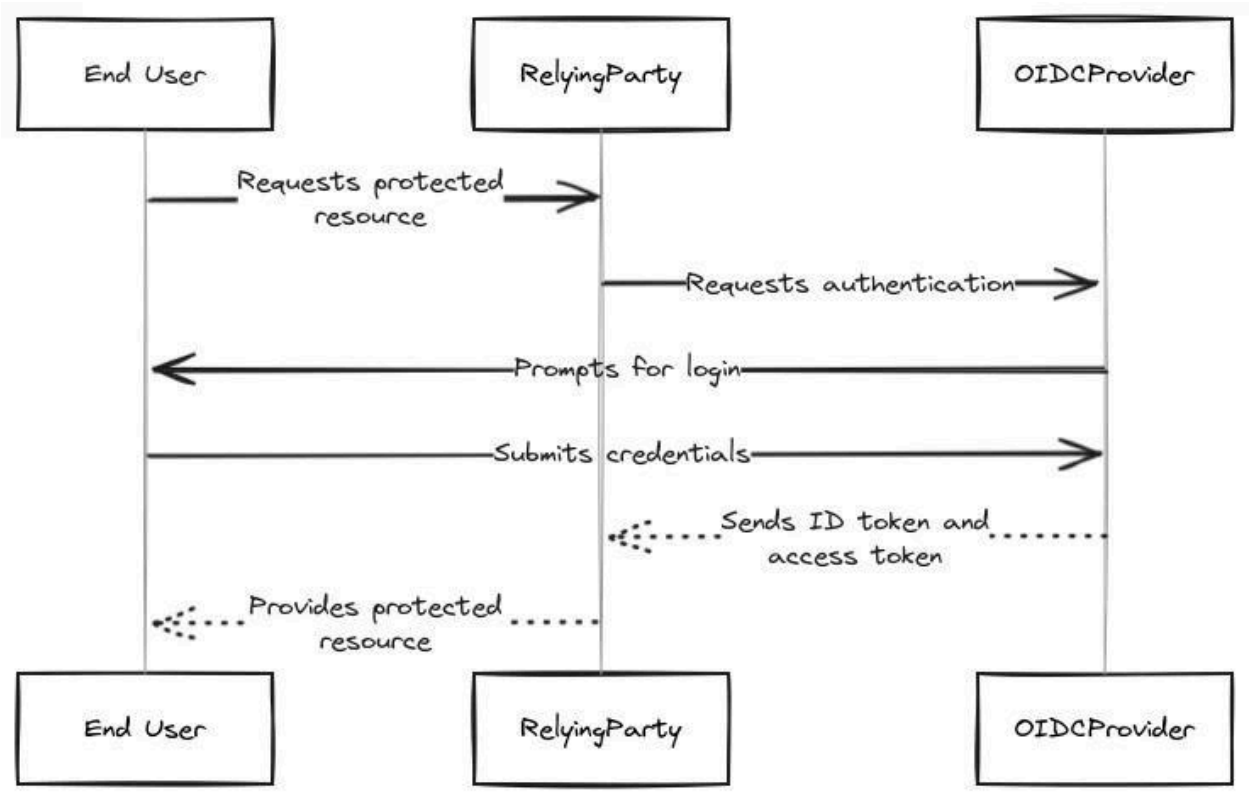
🏗️ OIDC Flow



Automate GitHub deployment with GitHub aws Actions







🚀 Step-by-Step Setup

1️⃣ Create OIDC Provider in AWS

Run once:

```
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1
```

2️⃣ Create IAM Role for GitHub

✅ Trust Policy (VERY IMPORTANT)

Replace:

- `<ACCOUNT_ID>`
- `<GITHUB_USERNAME>`
- `<REPO_NAME>`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated":
"arn:aws:iam::<ACCOUNT_ID>:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        },
        "StringLike": {
          "token.actions.githubusercontent.com:sub":
"repo:<GITHUB_USERNAME>/<REPO_NAME>:ref:refs/heads/main"
        }
      }
    }
  ]
}
```

3️⃣ Attach Permissions to Role

For your project:

```
{
  "Effect": "Allow",
  "Action": [
    "lambda:*",
    "apigateway:*",
    "dynamodb:*",
    "codedeploy:*",
    "cloudwatch:*",
    "logs:*",
    "iam:PassRole",
    "secretsmanager:*",
    "ec2:*"
  ],
  "Resource": "*"
}
```

👉 Later you can restrict this (least privilege)

4️⃣ Add Role ARN to GitHub

No secrets needed 👍

Just use it directly in pipeline:

role-to-assume: arn:aws:iam::<ACCOUNT_ID>:role/github-actions-role

---

5️⃣ Update GitHub Actions (Already Done)

✅ This is the key block:

```
- name: Configure AWS
  uses: aws-actions/configure-aws-credentials@v2
  with:
    role-to-assume: arn:aws:iam::<ACCOUNT_ID>:role/github-actions-role
    aws-region: us-east-1
```

6️⃣ Add Permissions in Workflow

At top of YAML:

```
permissions:
  id-token: write
  contents: read
```

👉 REQUIRED for OIDC

🧪 How to Verify It Works

Push code:

```
git push
```

Then in logs you should see:

```
Assuming role with OIDC
```

❗ Common Mistakes

❌ Wrong repo name

repo:user/repo

Must match exactly.

❌ Missing permissions

```
permissions:
  id-token: write
```

❌ Wrong branch

If using `main`, trust policy must match:

"ref:refs/heads/main"

❌ Using AWS keys + OIDC together

👉 Remove keys completely

🔐 Security Best Practices

Restrict to branch

"repo:your-org/your-repo:ref:refs/heads/main"

Restrict to environment (advanced)

"repo:your-org/your-repo:environment:prod"